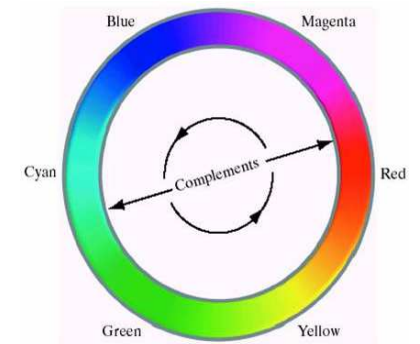
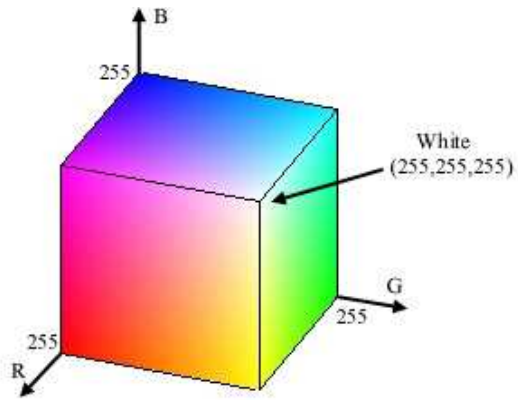
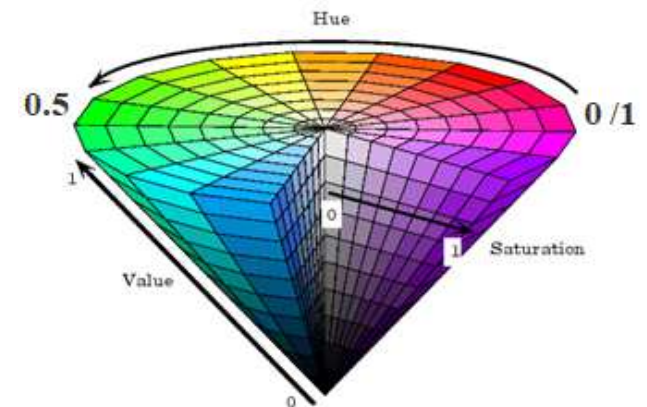
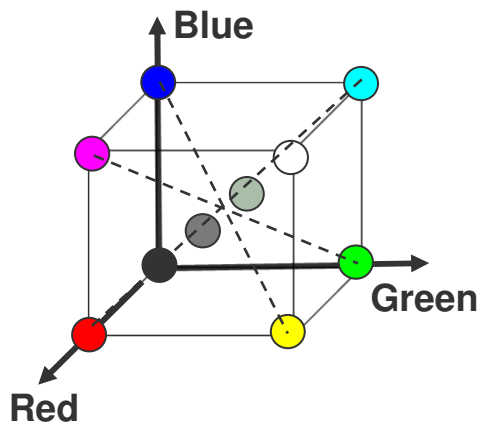




Software Engineering and Data Science

SEIS 766: Vision A.I.

Dr. Chih Lai



Basic Image Processing



■ Machine Learning in OpenCV, 7-Day Mini-Course

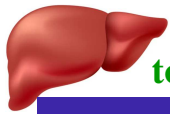
- https://machinelearningmastery.com/machine-learning-in-opencv-7-day-mini-course/?utm_source=drip&utm_medium=email&utm_campaign=Visualizing+missing+values+in+Ames+Housing&utm_content=Visualizing+missing+values+in+Ames+Housing

■ Getting started with Image Processing Using OpenCV

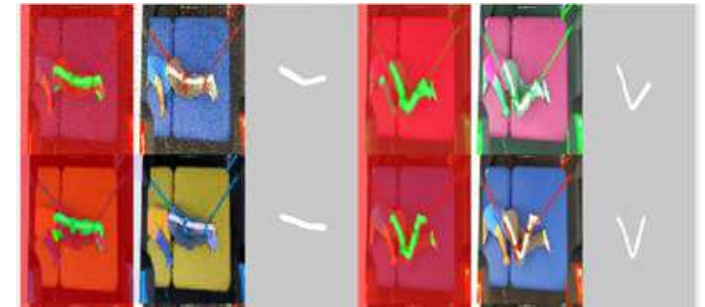
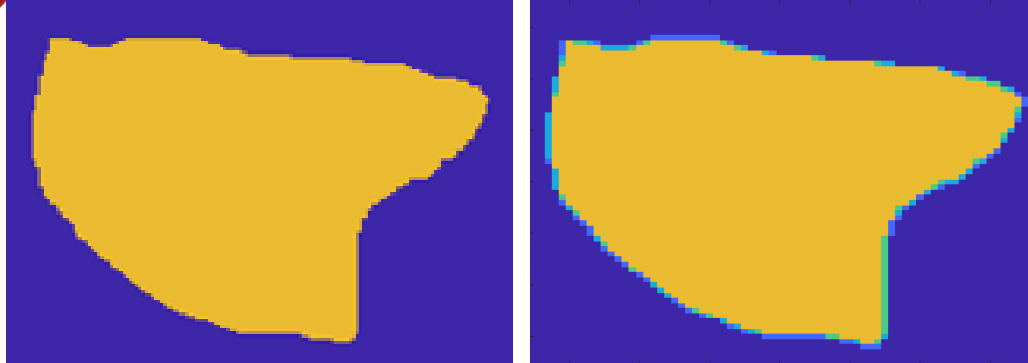
- <https://www.analyticsvidhya.com/blog/2023/03/getting-started-with-image-processing-using-opencv/>

“COLORs” to VAI as “Words” to Conv.AI

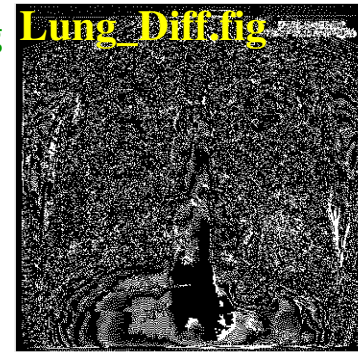
“COLORS” to VAI as “Words” to Conv.AI



toyobjects_resize.fig

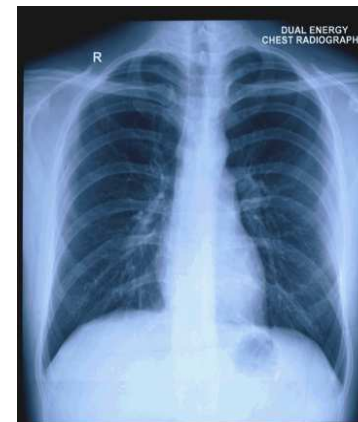


Lung_Compare.fig



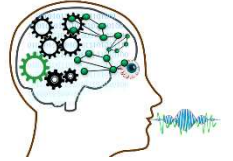
Lung_Diff.fig

skimage,
opencv,
pillow,
Matplotlib,
KerasCV

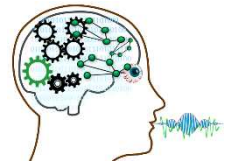


Lung_histeq.fig

Outline

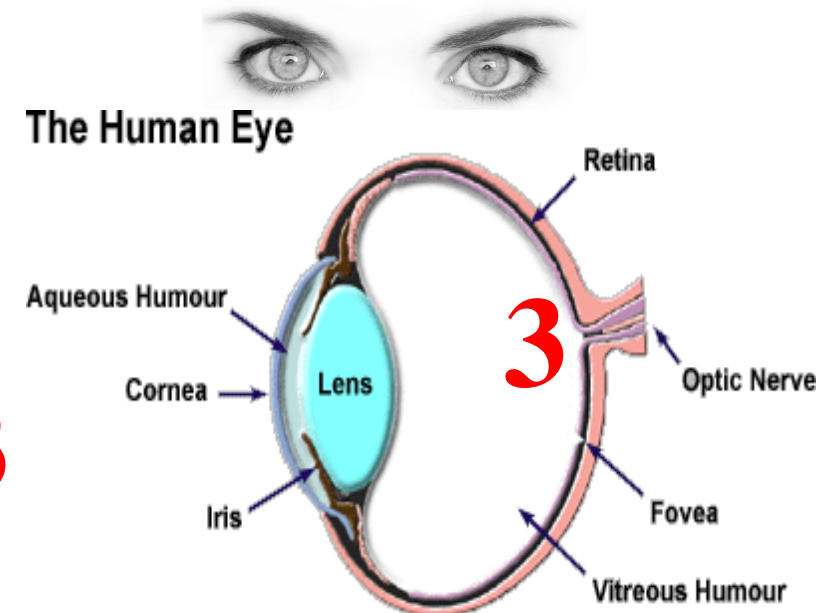
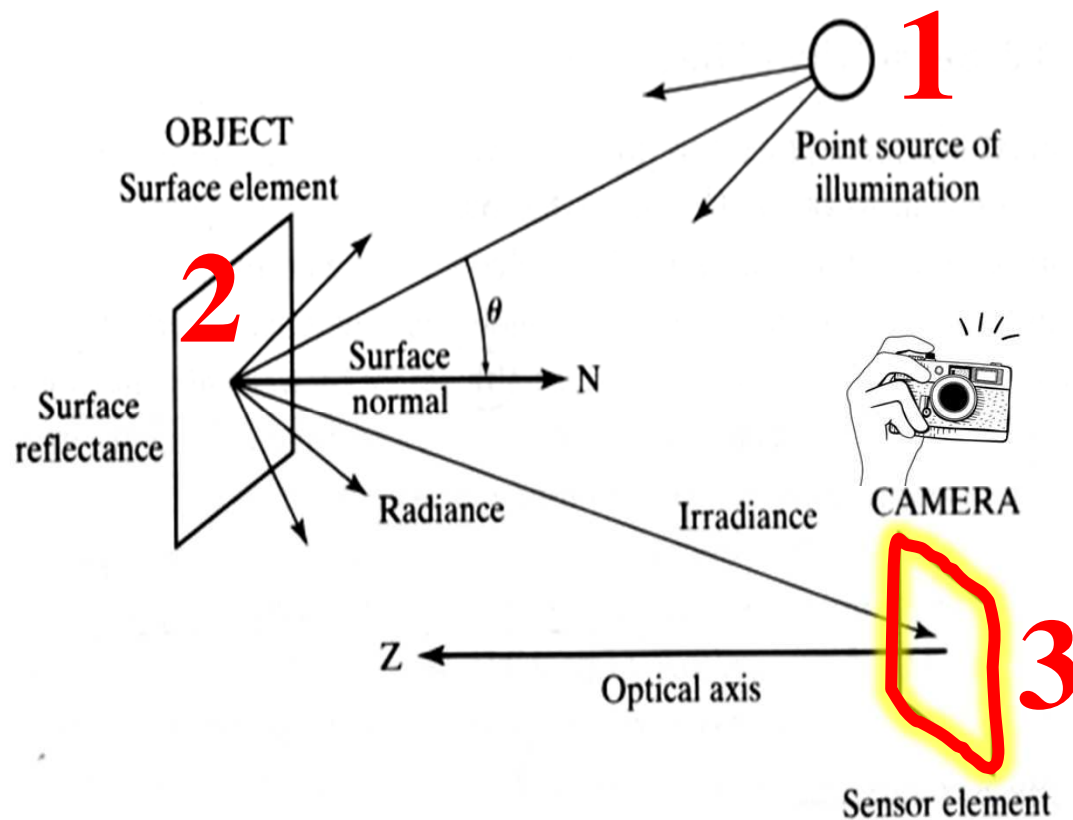


- → Color and Color Space
 - **RGB** Color Space, Gray-Scale, Black/White
 - Impact of image resizing to colors
 - Indexed Colors and Usage
 - Color Motion Map
 - Problems / Pitfalls in Dealing with Colors
 - HSV Color Space and Usage
 - Adjust Light Condition, Histogram Equalization



Perception of Objects

1. The spectrum (energy) of **light source**
2. The spectral reflectance of the **object surface**
3. The spectral sensitivity of the **sensor**



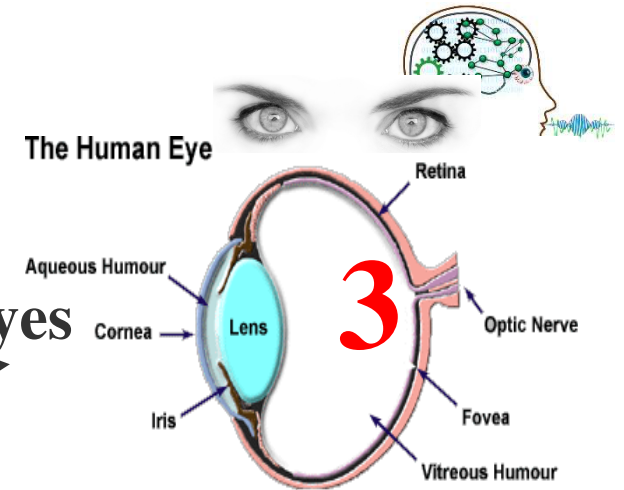
How Do We See An Object?

1 Light

2

Object

Eyes



- Luminance → Lightness
- Chrominance → Color

Any Difference?



RGB Images

1 Light

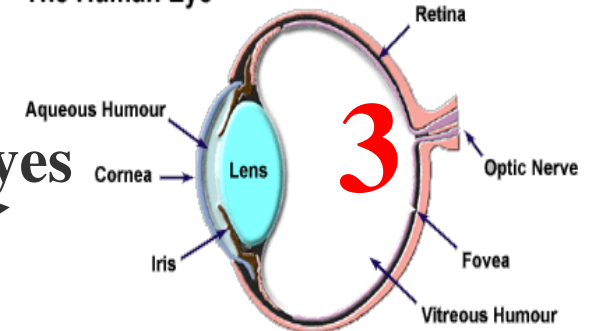
2

Object

Eyes

3

The Human Eye

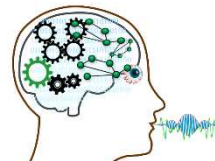


- Luminance → Lightness
- Chrominance → Color

Human eyes more sensitive to brightness than to colors
BUT, RGB images have **NO** specific light information.

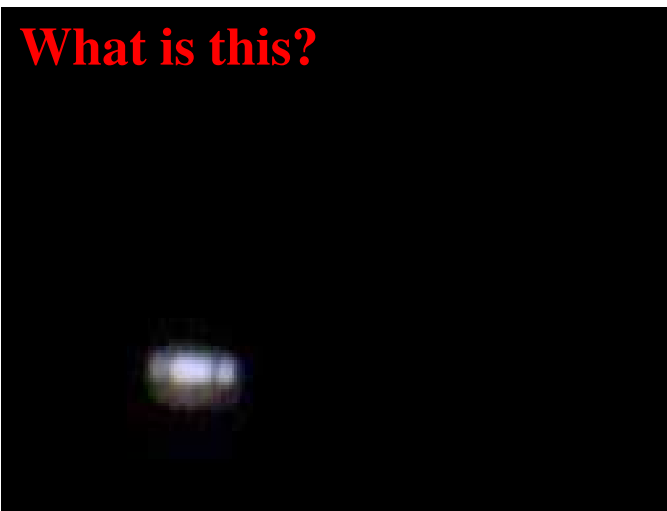
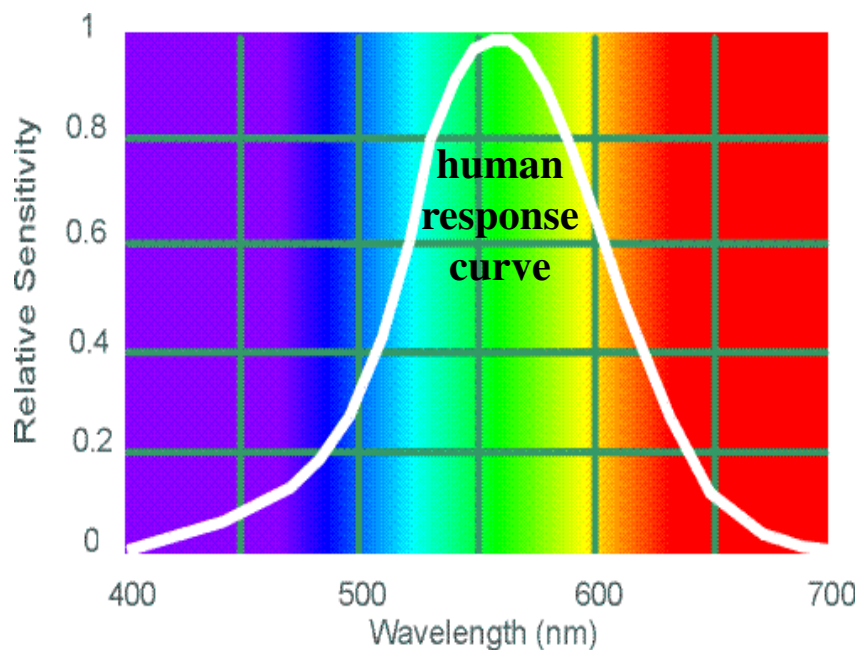
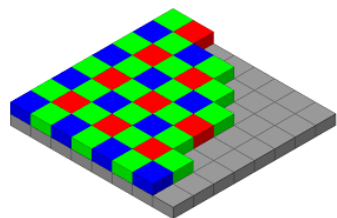
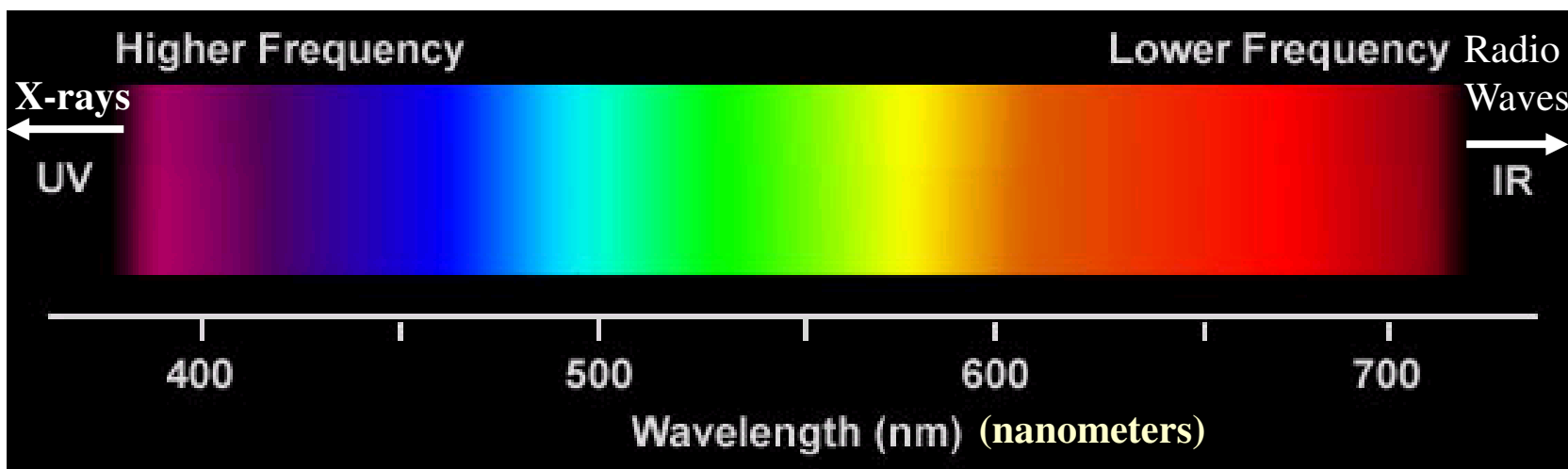
Any Difference?



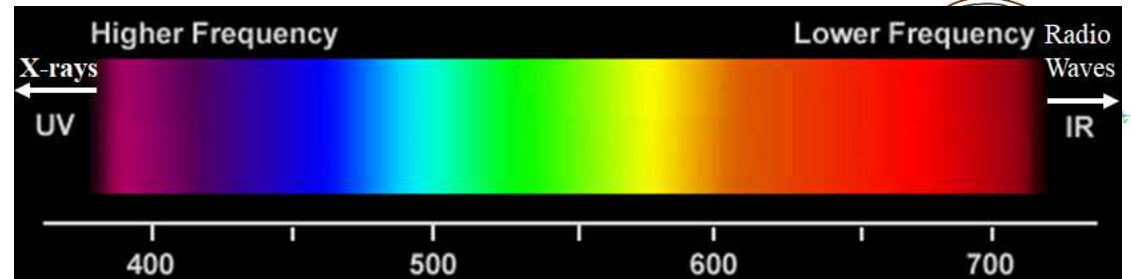


Visible Light Spectrum

Color is perceived as light-wave 400nm to 700nm received at retina



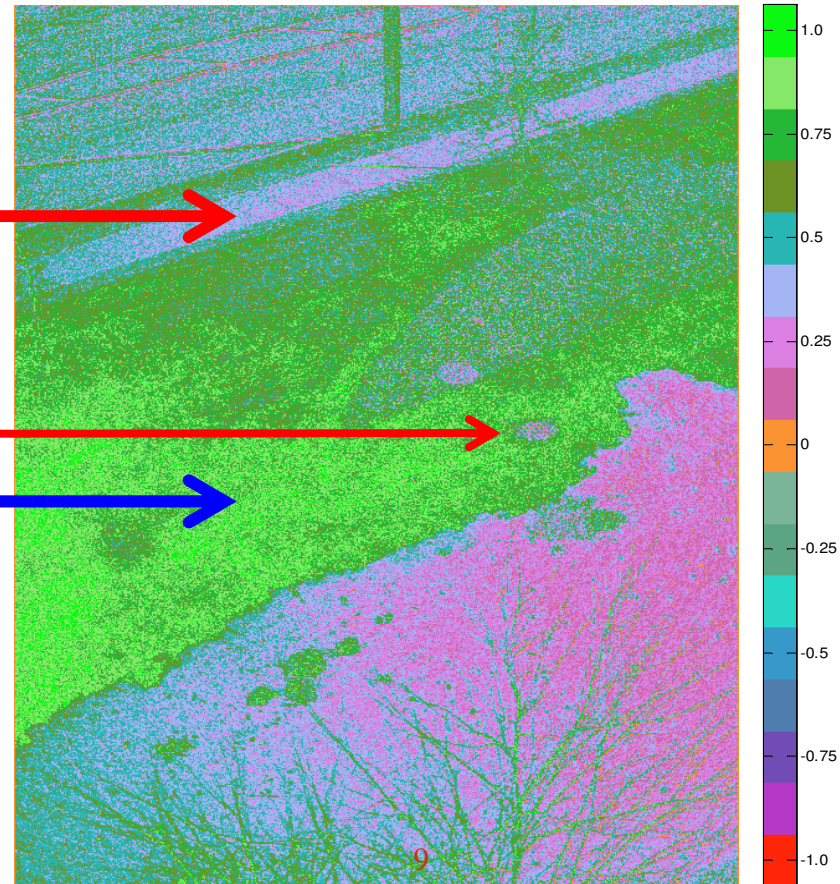
Color Usage Combine **RGB** & Infrared to NDVI Images



RGB Image



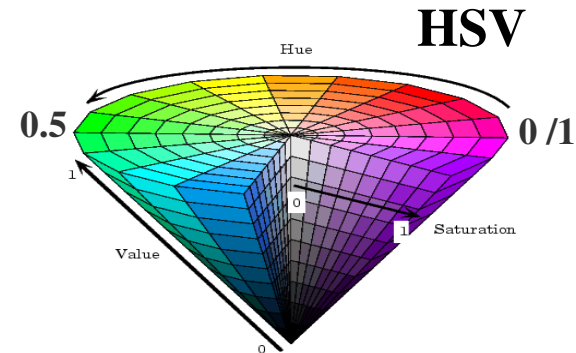
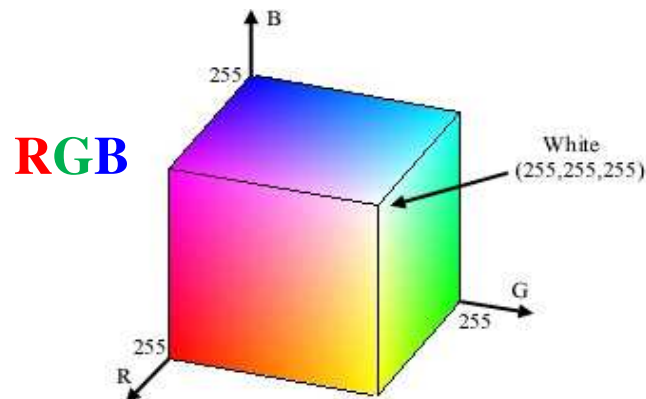
NDVI Map $NDVI = \frac{NIR - R}{NIR + R}$
(Normalized Difference Vegetation Index)



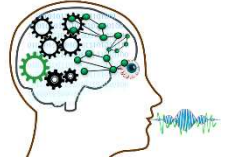


Color Space / Color Models = Represent Colors for Computation

- For representing and mapping between computable color space(sssss)
 - Different ways to **model** / **represent** colors
 - For example, **RGB**, gray scale, black / white images
 - Some common color space: **RGB**, HSV, YCbCr, CMY, CMYK, YIQ
 - Another color space: **indexed colors**

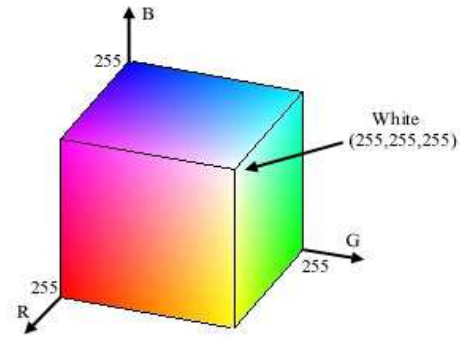


Common Color Models



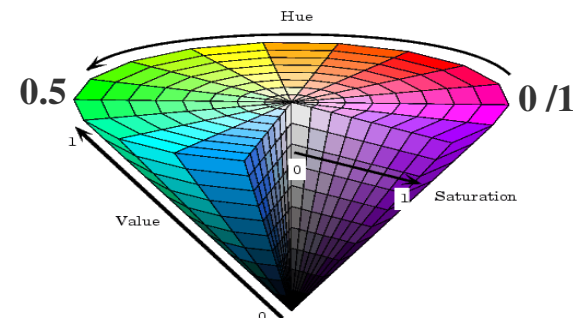
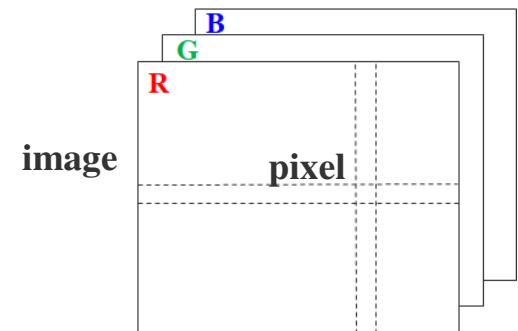
■ Hardware oriented color models

- **RGB** (Red, Green, Blue) model
 - For color monitors
- CMY (Cyan, Magenta, Yellow) model
 - For color printers
- YIQ (Luminance, Inphase, Quadrature) model (also refer as **NTSC model**)
 - For color television broadcasting

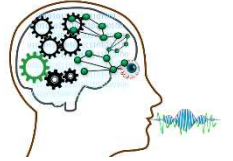


■ Application / computation oriented color models

- **RGB** (Red, Green, Blue) model
- **HSI** (Hue, Saturation, Intensity) model
 - Nonuniform color space
- L*a*b* model
 - Uniform color space



Outline



- Color and Color Space

- **RGB** Color Space, Gray-Scale, Black/White

- Impact of **image resizing** to colors

- Indexed Colors and Usage

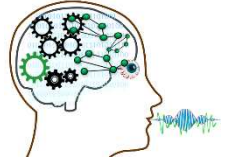
- Color Motion Map

- Problems / Pitfalls in Dealing with Colors

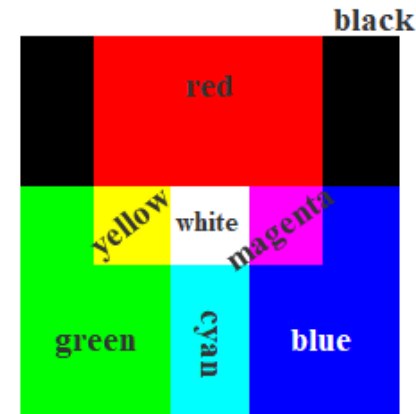
- HSV Color Space and Usage

- Adjust Light Condition, Histogram Equalization

RGB Model

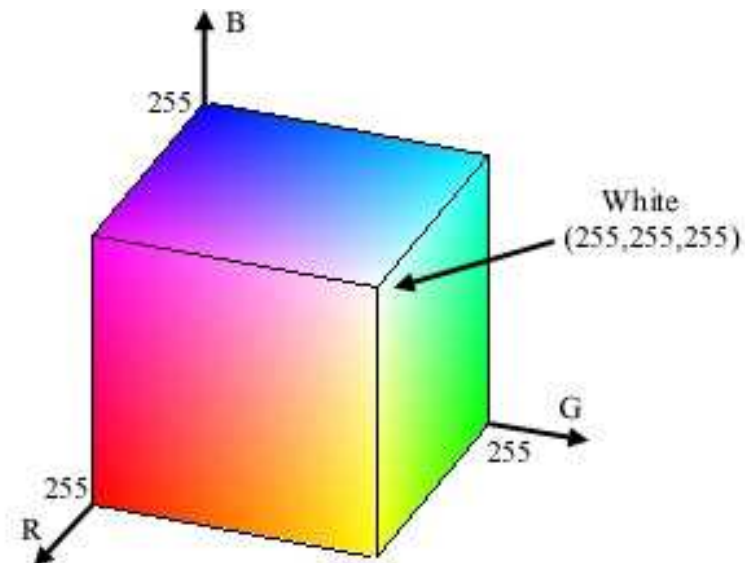
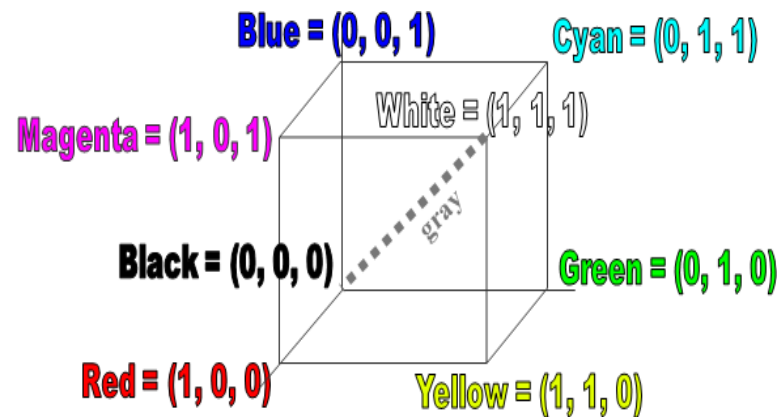


- Different intensities of **red**, **green**, and **blue**
 - Combine together to generate various colors
 - BUT... **NO** information specifically for *light*



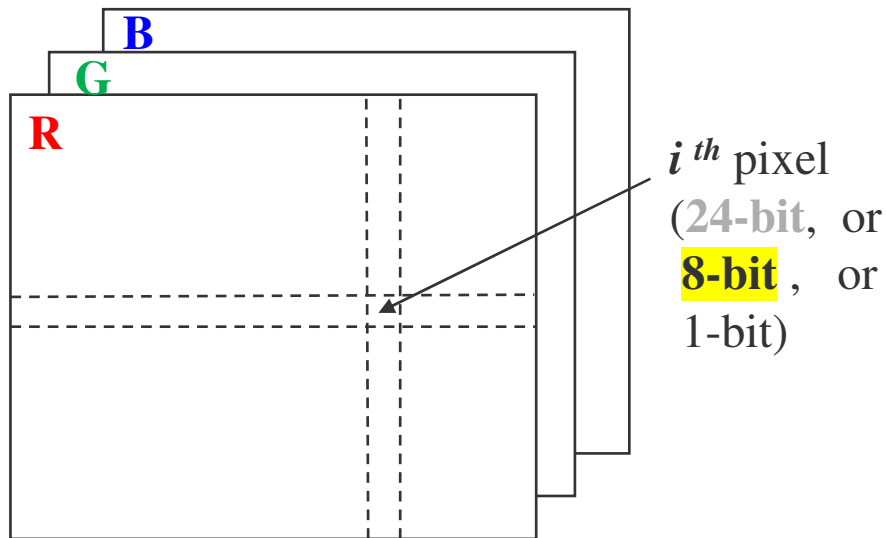
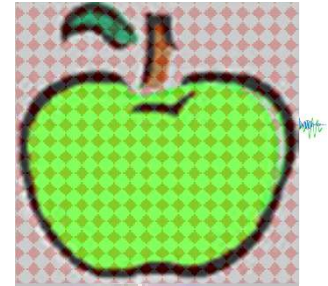
★ Colors = mixture of **R**, **G**, **B** values of **2** different types

1. **Double / floating-point values** between **0.0** (none) and **1.0** (lots)
2. **Unsigned 8-bit integer values** between **0** (none) and **255** (lots)



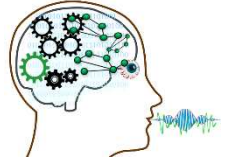
Images and Colors

- *Images, pictures* are represented by *bitmaps* (*Raster Image*)
- A bitmap is a **2** or **3**-dimensional matrix (array) of *pixels*
 - Each pixel has a numerical value called *intensity* or *amplitude*
 - Each pixel may be associated with 3D values (r, g, b)
 - The number of bits available to code a pixel is called *pixel depth*



How about this???

Pixel Data Types



■ unsigned integer

- $0 \dots 2^8 - 1 = 0 \dots 255$ (usually 8-bit per pixel)
- Each pixel ranges from 0 .. 255

%% produce output using “uint8”

```
i = uint8(zeros(30, 256));           %% np.uint8()
for x = 1 : 256, i(:, x) = x - 1; end; % assign 0:255
imshow(i)
```

%% next, produce same output using “double”

```
i = zeros(30, 256);                 %% double
for x = 1 : 256, i(:, x) = (x-1) / 255; end; % assign 0:1
imshow(i)
```

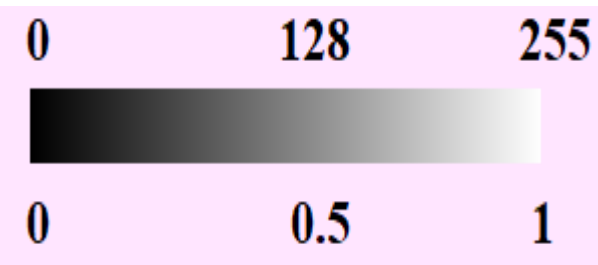
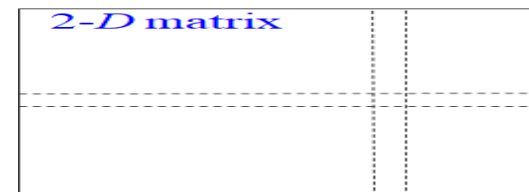
```
i_u8 = np.zeros((30, 256), dtype=np.uint8)
for x in range(256):                # columns x = 0..255
    i_u8[:, x] = x                   # assign 0:255
```

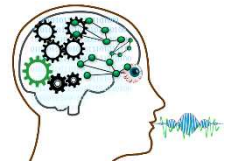
```
i_f = np.zeros((30, 256), dtype=np.float64)
for x in range(256):                # columns x = 0..255
    i_f[:, x] = x / 255.0           # assign 0..1
```

VAI_Slide_uint8_float.ipynb

■ double / floating-pt

- Each pixel ranges from 0 .. 1





Exercises: Data Types and Colors

VAI_Slide_ColorTypes_Exercise.ipynb

```
import matplotlib.pyplot as plt
b = np.zeros((120, 120, 3), dtype=np.float64) # 1.
print(type(b), b.dtype)
plt.imshow(b)      plt.show()

b1 = np.ones((120, 120, 3), dtype=np.float64) # 2.
b2 = b1.astype(np.uint8) # type cast to uint8 type 3.
```

uint8	0	128	255
			
float / double	0	0.5	1

VAI_Slide_ColorTypes_Exercise.m

```
b = zeros(120,120,3); % 1.
class(b)                % double / floating-pt
figure, imshow(b)       % black or white???

b1 = ones(120,120,3); % double 2.
figure, imshow(b1)      % black or white???

b2 = uint8(b1);        % cast to uint8 type 3.

figure, imshow(b2)      % black or white???

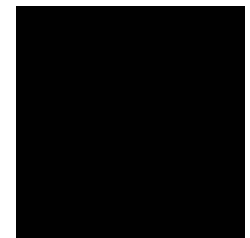
figure, imshow(b2*255)  % black or white???
```

b = All zeros

b1 = All ones

b2 = uint8(**b1**)

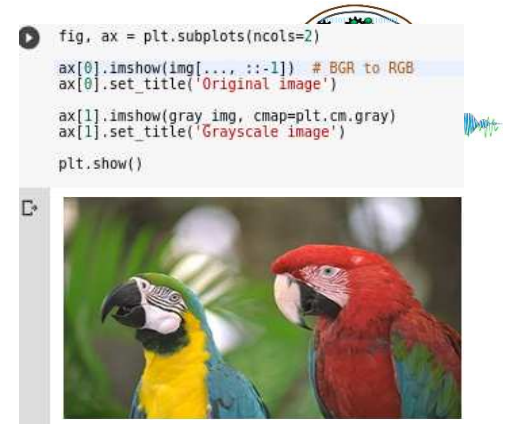
= 255 (data type?)



Color to Gray

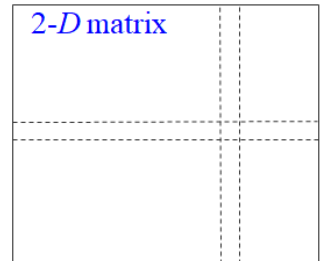
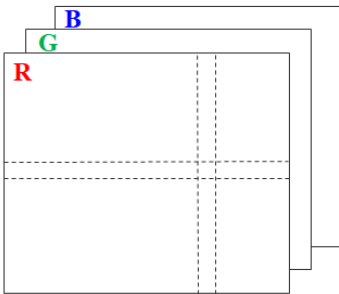
```
i = imread('parrots.jpg');
imshow(i)
g = rgb2gray(i);
```

```
import cv2 # OpenCV
i = cv2.imread('parrots.jpg')
cv2.imshow(i)
cv2.cvtColor(i, cv2.COLOR_BGR2GRAY)
```



$H \times W \times 3$

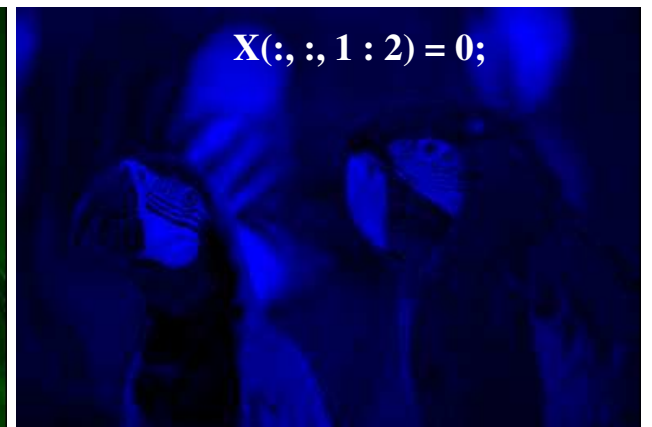
$H \times W$



$X(:, :, 2 : 3) = 0;$



$X(:, :, [1 \ 3]) = 0;$



$X(:, :, 1 : 2) = 0;$

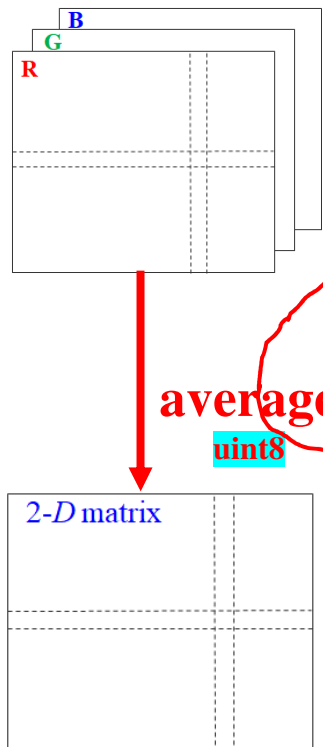


Color to Gray, Use “average” Function or Compute Yourself

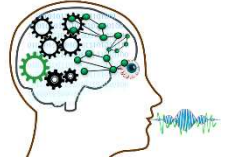
```
i = imread('parrots.jpg'); % cv2.imread
g = rgb2gray(i); % cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
m = uint8(mean(i, 3)); % np.mean(image, axis=2).astype(np.uint8)
a{1} = i; a{2} = g; a{3} = m; a{4} = imabsdiff(g, m);
figure, montage(a) % NO easy montage, in cv2.imshow
```

uint8	0	128	255
float / double	0	0.5	1

rgb2gray()



Convert To Binary Images, WHY??



- Convert gray image into “black / white”

**Input
Gray
Image**



B&W



```
im2bw(image)
```

```
cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
cv2.threshold(im_gray, 128, 255, cv2.THRESH_OTSU)
```

WHY to Black & White???

Visualization

Object ID foreground vs background

For quick training on smaller CNNs

Document OCR recognition

Morphological Operations



Convert To Binary Images

`im2bw(image)`

`cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

`cv2.threshold(im_gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)`

■ Use `im2bw()` function

- **1.** First, convert an **RGB** to a grayscale image
- **2.** The grayscale image is then **thresholding** to binary image
- Gray-scale value < *threshold* → 0 (black) otherwise → 1 (white)
- W/O specified threshold, `im2bw()` auto-decide a threshold using Otsu's method `graythresh()`

```
i = imread('Hmattface.jpg');
```

```
size(i)
```

```
% ans = 256 256 3
```

```
imshow(im2bw(i, 0.3))
```

```
imshow(im2bw(i, 0.1))
```

```
size(im2bw(i, 0.3))
```

```
% ans = 256 256
```

```
imshow(im2bw(i)) % no TH
```

```
graythresh(i)
```

```
% % OTSU = 0.4039
```



grayscale
image



B&W



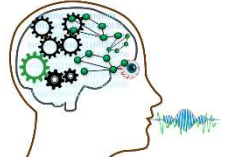
0.4039

0.3

0.1

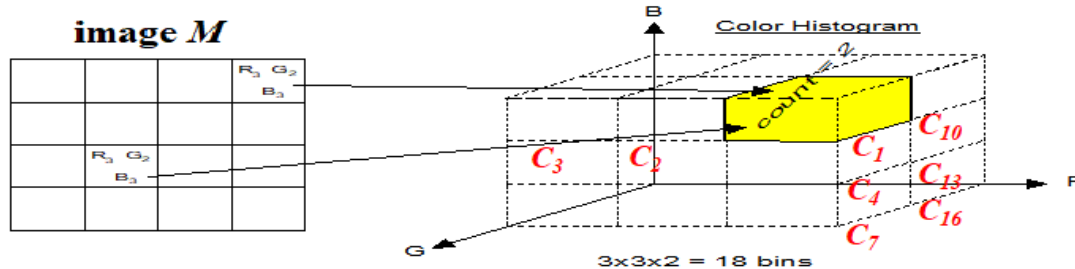
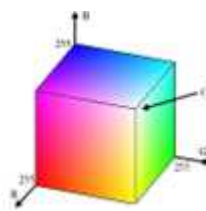
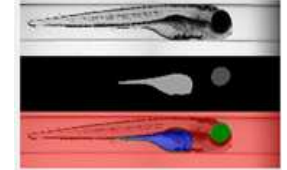
Threshold to white >

Outline



- Color and Color Space
- **RGB** Color Space, Gray-Scale, Black/White
 - Impact of image resizing to colors
-  Indexed Colors and Usage
 - Color Motion Map
- Problems / Pitfalls in Dealing with Colors
- HSV Color Space and Usage
 - Adjust Light Condition, Histogram Equalization

-

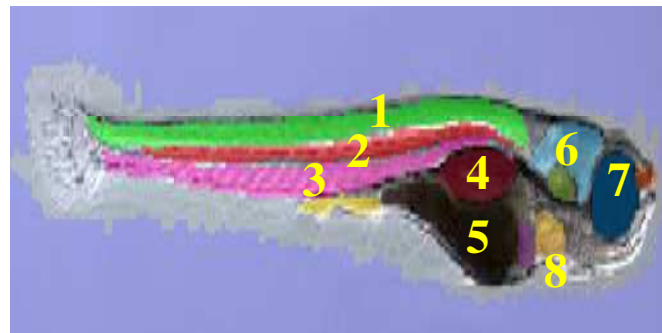


***m* = 18**
index map
for 18-color

R	G	B
0.3	0.2	0.6
.....		

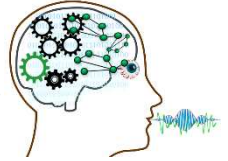


8 colors

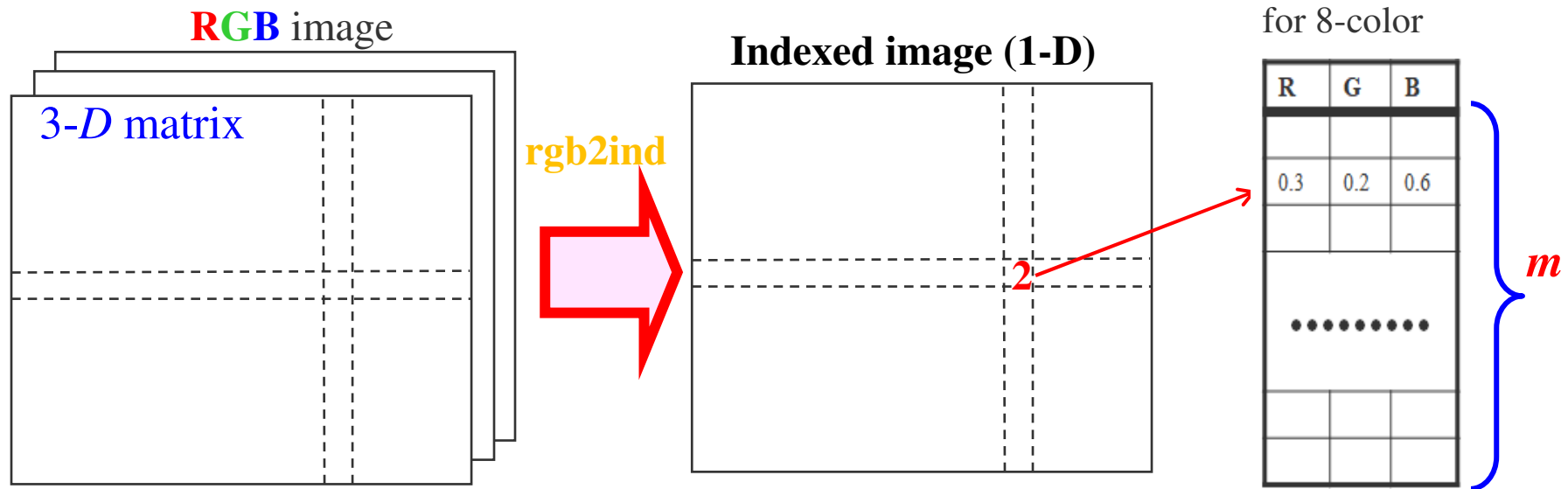


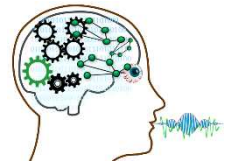
- © Copyright 2026 by Dr. Chih Lai.

Color Reduction Using Indexed Colors In Indexed Images



- An indexed image contains **2** components:
 - An indexed **image** matrix, and an indexed **color map** of size $m \times 3$
 - m is the number of colors
- Each pixel in an original image is mapped to an indexed image
 - `[idx_image, map] = rgb2ind(image, num_colors);`
 - `image.quantize(colors = num_colors)` **##** ← this is from **PIL** package, **NOT** OpenCV
 - Saving / storing “**map**” with the indexed image





Converting to Indexed Colors

```
Idx_img = image.quantize(colors = num_colors)  ## ← this is from PIL package
```

```
map = Idx_img.getpalette()
```

```
[idx_image, map] = rgb2ind(image, num_colors);
```

```
### OR
```

```
### numpy package
```

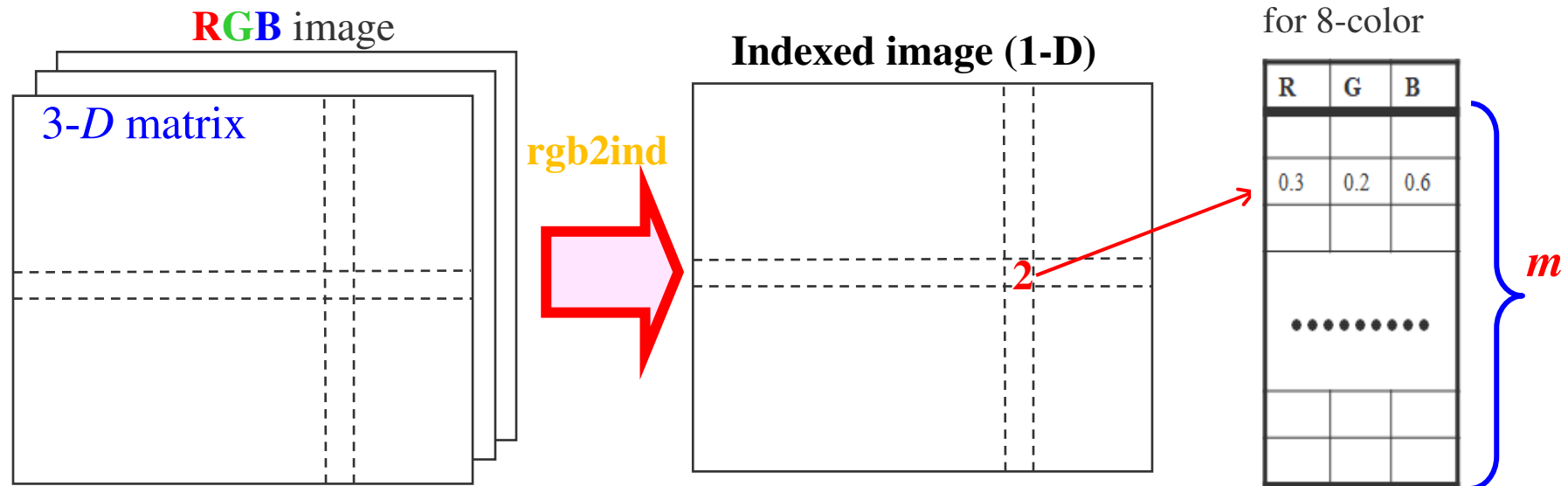
```
# Define custom index colormap
```

```
color_index = np.unique(Img_RAW)  # Finds all distinct pixel values
```

```
color_index_map = {color: index for index, color in enumerate(color_index)}
```

```
# Convert to index-color image
```

```
Img_indexed = np.vectorize(color_index_map.get)(Img_RAW)
```



Color Reduction

VAI_Slide_IdxCOLOR. m / ipynb

```
i = imread('parrots.jpg');
```

```
max(i(:)), min(i(:))
```

```
% ans = 255 0
```

```
figure, subplot(1,2,1)
```

```
imshow(i);
```

```
[x, map] = rgb2ind(i, 256);
```

```
size(map)
```

```
% ans = 256 3
```

```
[x, map] = rgb2ind(i, 64);
```

```
subplot(1,2,2)
```

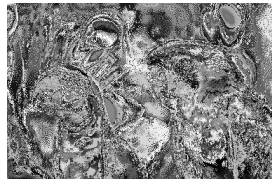
```
imshow(x, map);
```

```
colormap(map); colorbar
```

```
imshow(x)
```

```
imtool(x)
```

```
imtool(x, map)
```



$2^{24}=16$ million colors



256 colors



64 colors



8 colors



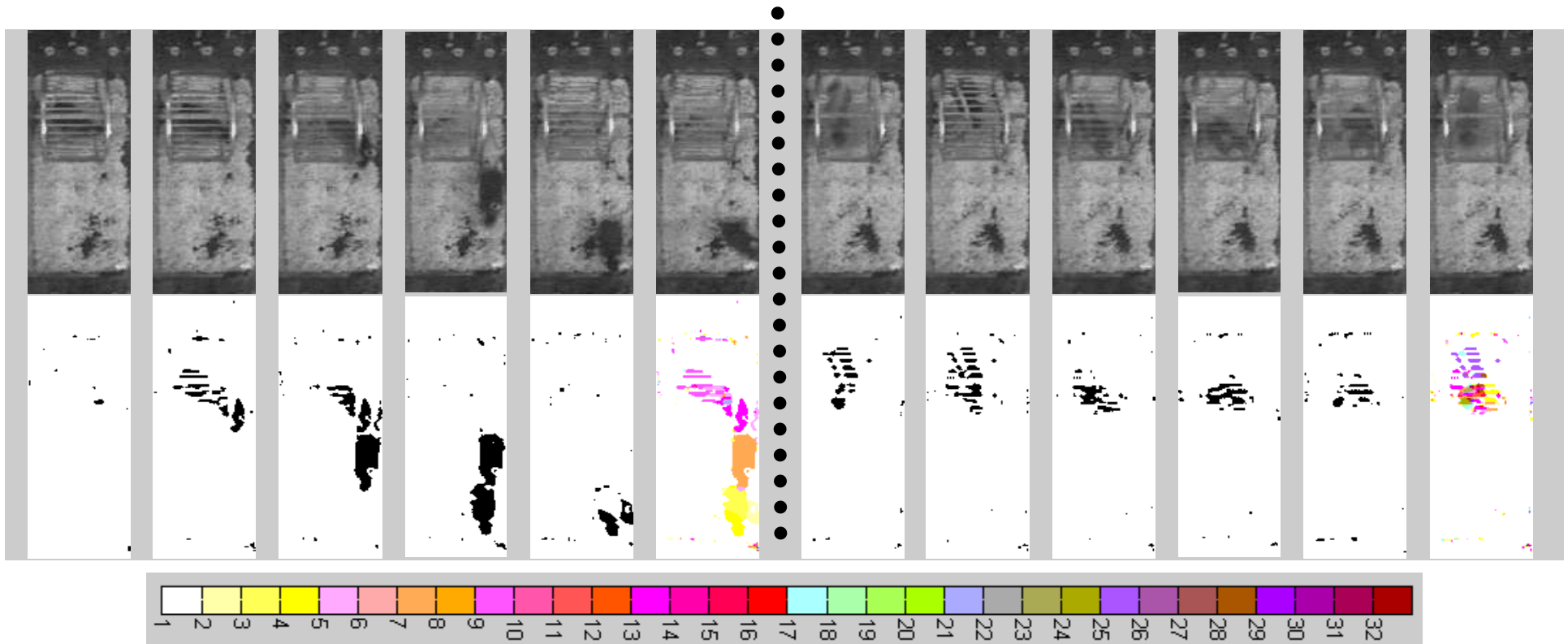
4 colors



2 colors

Build Indexed **COLOR** Images from Movements = **Motion Maps**

1. Save memory & computing / visual
2. Auto-labeling / mask conversion
3. Others, i.e. represent temporal info in motion

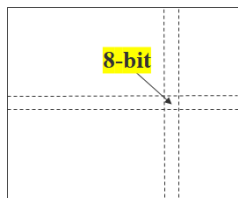


each bit

16	8	4	2	1

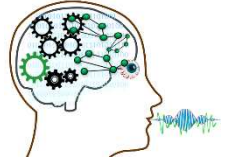
2^5 colors $\therefore w = 5$

<https://dl.acm.org/doi/10.1145/1233321.1233322>



from **PIL** import Image

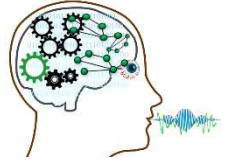
image.convert('P', palette=Image.ADAPTIVE, colors=32, dither=Image.NONE)



Appendix

- Building Motion Map from Indexed **COLORs**
- <https://dl.acm.org/doi/10.1145/1233321.1233322>

Binary Motion Maps (*BMMs*) in Black and White

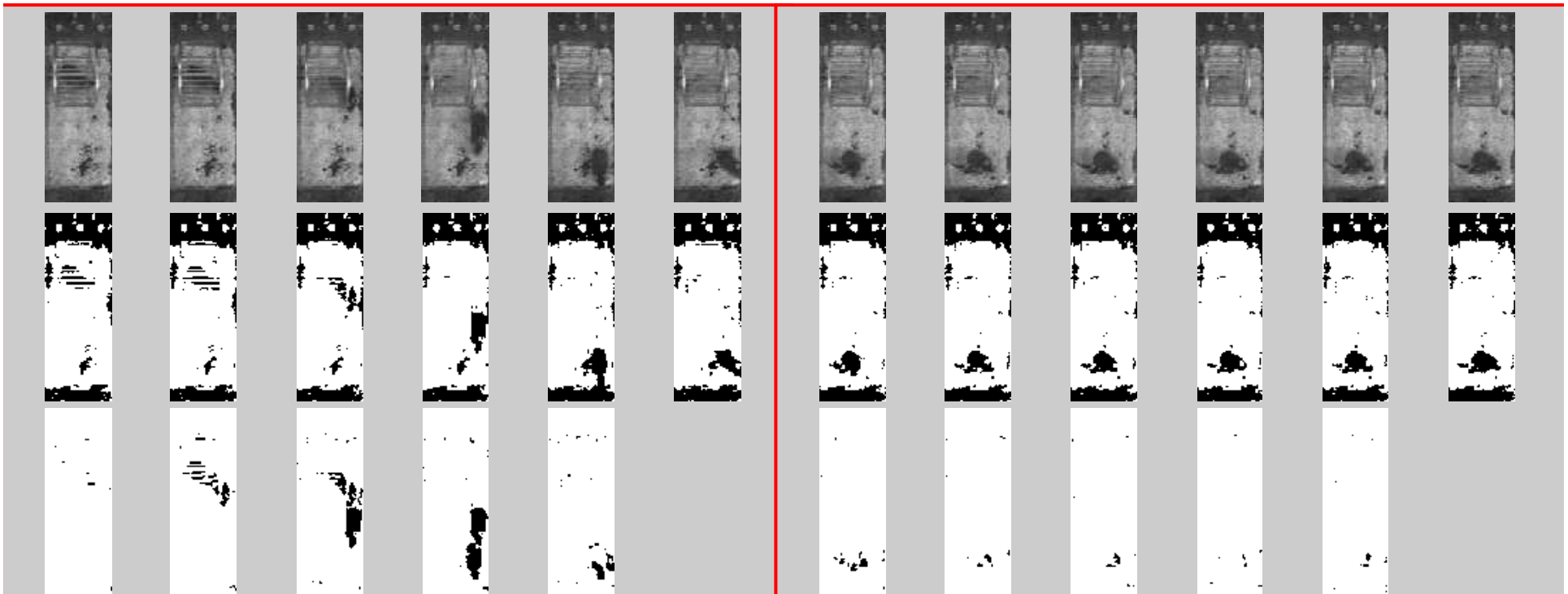


- Converted each gray-scale image into a black-and-white (0-and-1) image
- Animal's spatial movement over two video frames can be obtained by an XOR

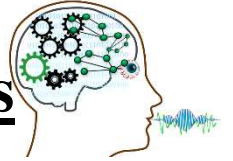
$$BMM_t(r, c) = I_t(r, c) \oplus I_{t+1}(r, c)$$



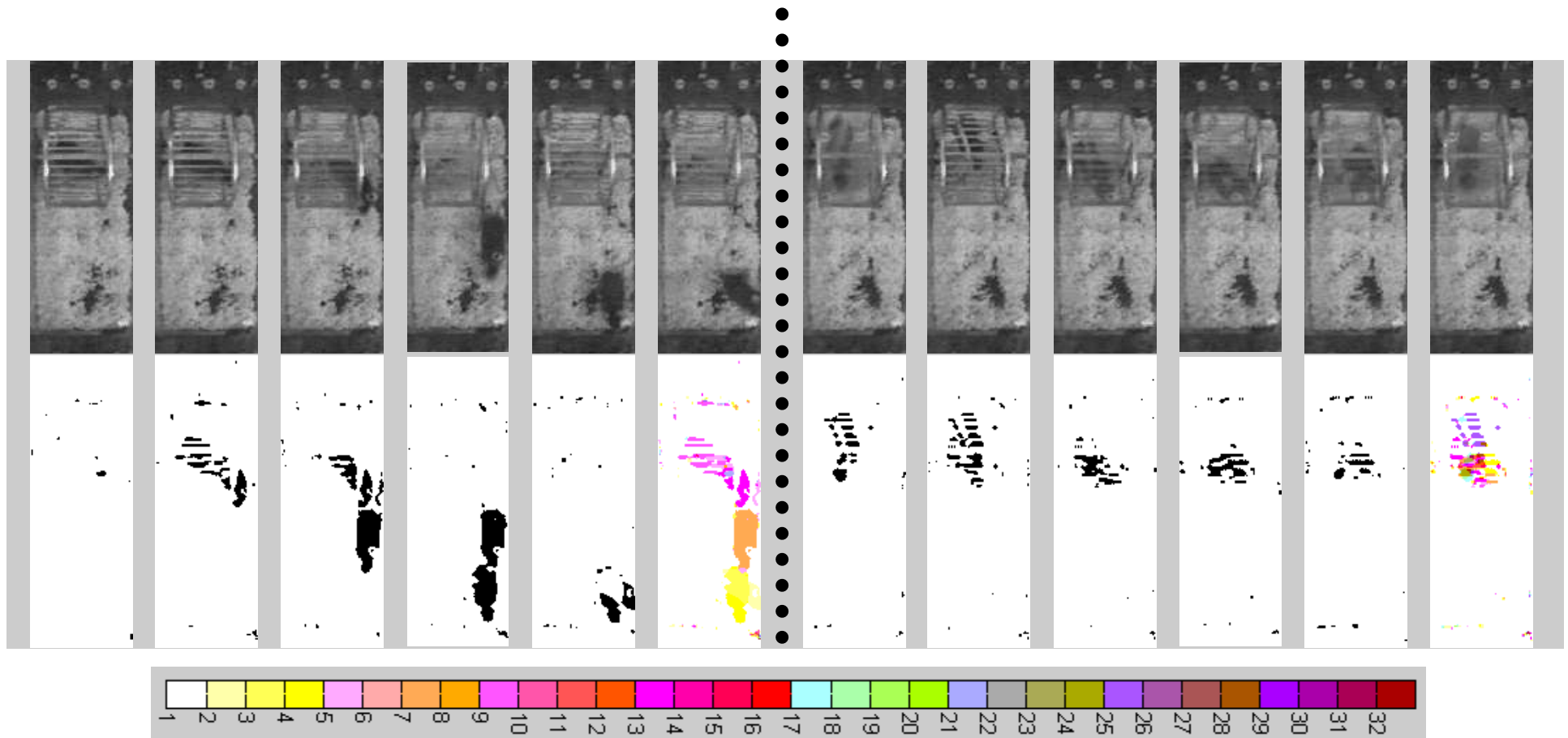
- Producing a *binary motion map* (*BMM*)



Different Movements Different COLOR Motion Maps



- Video frames taken 400 milliseconds apart with $w = 5$ over two seconds



Represent Motion in Colors



- Video taken 400 ms apart

